

Notes 8: Dimension Reduction

Table of contents

Introduction	149
Principal Component Analysis	149
Partial Least Squares	152
PLS Regression in R	152
<code>plsrf</code> Function Output Interpretation	153
More Differences Between PCA and PLS	153
PLS Algorithm	154
PLS Surrogates	155
Cross Validation in PLS	156
Cross Validation in PLS with <code>Caret</code>	157
Fitting Final PLS Model	157
UMAP	159
UMAP in R	159
UMAP 3D Example vs PCA	161

Introduction

It is often that case when working with data that there are many features or predictors we need to deal with. Sometimes you can reduce the number of variables with domain knowledge: knowing what variables should not be in the analysis. However, even that fails sometimes when there are hundreds or even thousands of features in the dataset. In this case, dimension reduction techniques are needed.

There are two main goals of dimension reduction methods:

1. Reducing the number of features for use in a model.
2. Reducing the number of features for visualization.

Sometimes both goals are desired, but other times only one is needed. In this set of notes, we will discuss three dimension reduction techniques: principal component analysis (PCA), partial least squares (PLS), and uniform manifold approximation and projection (UMAP).

Principal Component Analysis

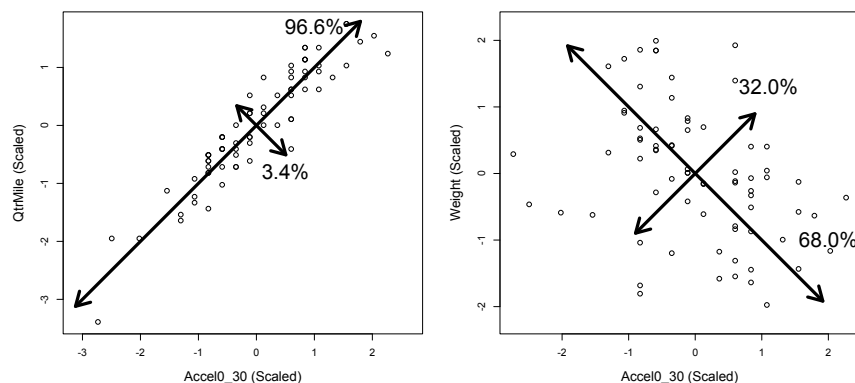
Principal component analysis (PCA) is probably the most popular dimension reduction technique. It is useful for both modeling and visualization. Let's focus first on the modeling and we will discuss the visualization when we discuss UMAP.

The idea of PCA is to obtain k new predictors, called components, with $k < p$ where each new predictor is a linear combination of the original predictor variables. PCA is useful for:

- Reducing the dimension of the problem so we are using k predictors instead of p predictors.
- Reducing problems associated with multicollinearity.

- Slopes being hard to interpret.
- Standard errors being inflated.

The way PCA works is by creating orthogonal predictor components using information from all the original predictors. The first component explains as much of the variation (or information) as possible in the original explanatory variables. The second component then explains as much as possible of the remaining variation, and so on. The more correlated the original variables are, the more variation can be explained with each new linear combination.



Those images came from the PCA on the `cars99` dataset that is in the `math3190package` and in the `Math3190_Sp26` GitHub repo.

```
library(math3190package)
cars <- na.omit(cars99)
head(cars)
```

```
# A tibble: 6 x 10
  Model      CityMPG HwyMPG FuelCap Weight FrontWt Accel0_30 Accel0_60
  <chr>      <int>  <int>   <dbl> <int>  <int>    <dbl>    <dbl>
1 "Buick Ce~    20    29   17.5  3350    64     3.3     9.4
2 "Buick Re~    19    30   17.5  3325    63     3.3     9.2
3 "Chevrole~    20    29   16.6  3350    64     3.4     9.8
4 "Chevrole~    22    30   15     3040    64     3.3     9.4
5 "Chrysler~    19    27   16     3170    64     3.5    10
6 "Daewoo L~    20    28   15.8  3185    60     4      11

# i 2 more variables: QtrMile <dbl>, Type <chr>
```

We can find the components using the `prcomp()` function in **R**. Let's use PCA to try to predict highway mpg using fuel capacity, weight, front weight, 0-30 mph acceleration time, 0-60 mph acceleration time, and quarter mile time. Since the scale is so vastly different between the variables, we will center and scale them.

```
X <- cars[,4:9] # numeric predictors are in columns 4-9
y <- cars$HwyMPG
pca <- prcomp(X, scale = T)
```

The amount of variation explained by each component can be found using the standard deviations of the components. We want to balance taking as many components as we need to still explain the vast majority of variation in the original variables with reducing the dimension of our feature space. A common rule of thumb is to take as many components as we need to explain at least 90% of the

variation.

```
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6
Standard deviation	1.9414	1.1471	0.8395	0.34991	0.2877	0.07364
Proportion of Variance	0.6281	0.2193	0.1174	0.02041	0.0138	0.00090
Cumulative Proportion	0.6281	0.8474	0.9649	0.98530	0.9991	1.00000

The rule of thumb

```
round(pca$rotation, 3) # how the components are created
```

	PC1	PC2	PC3	PC4	PC5	PC6
FuelCap	-0.349	0.597	-0.127	0.681	0.203	-0.028
Weight	-0.345	0.602	-0.140	-0.698	-0.104	0.023
FrontWt	0.268	0.353	0.896	-0.014	0.033	0.001
Accel0_30	0.461	0.287	-0.219	0.182	-0.789	0.045
Accel0_60	0.486	0.193	-0.240	-0.069	0.444	0.683
QtrMile	0.487	0.196	-0.237	-0.102	0.357	-0.728

These components are related to how correlated the predictor variables are. The way we find these components is by finding the eigenvectors of the correlation matrix (for scaled PCA) or the covariance matrix (for un-scaled). Then the standard deviations are the square roots of the eigenvalues.

```
eig <- eigen(cor(X))
```

```
sqrt(eig$values) |> round(3)
```

```
[1] 1.941 1.147 0.839 0.350 0.288 0.074
```

```
eig$vectors |> round(3)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]
[1,]	-0.349	-0.597	-0.127	0.681	-0.203	0.028
[2,]	-0.345	-0.602	-0.140	-0.698	0.104	-0.023
[3,]	0.268	-0.353	0.896	-0.014	-0.033	-0.001
[4,]	0.461	-0.287	-0.219	0.182	0.789	-0.045
[5,]	0.486	-0.193	-0.240	-0.069	-0.444	-0.683
[6,]	0.487	-0.196	-0.237	-0.102	-0.357	0.728

Once we have these components, found in `pca$x`, we can put those into a model like an OLS regression or plot them to visualize the higher-dimension space projected down to 1 or 2 dimensions.

Partial Least Squares

Notice that the response variable, y , is never used to find the components in PCA. Because of this, PCA is known as an **unsupervised** statistical learning algorithm.

However, a natural idea is to do something similar to PCA, but to make the algorithm **supervised**. That is, we can try to find these components using information from $\mathbf{y}$ in addition to $\mathbf{X}$. This is essentially what **partial least squares** (PLS) is. PLS is a **dimension reduction** tool similar to PCA that finds these components by balancing explaining variation in the predictors and correlation with the response variable.

PLS Regression in R

We can use the `plsr()` function in the `pls` package to perform partial least squares regression. There is also a `pcr()` function to implement PCA.

```
library(pls) |> suppressPackageStartupMessages()
cars_pls <- plsr(HwyMPG ~ FuelCap + Weight + FrontWt +
                 Accel0_30 + Accel0_60 + QtrMile,
                 data = cars, scale = TRUE)
cars_pcr <- pcr(HwyMPG ~ FuelCap + Weight + FrontWt +
                 Accel0_30 + Accel0_60 + QtrMile,
                 data = cars, scale = TRUE)
```

Let's compare the output of the `pcr()` function to our PCA.

```
summary(cars_pcr)
Number of components considered: 6
TRAINING: % variance explained
```

	1 comps	2 comps	3 comps	4 comps	5 comps	6 comps
X	62.81	84.74	96.49	98.53	99.91	100.00
HwyMPG	38.86	56.53	66.22	71.13	71.14	71.87

```
round(cars_pcr$projection, 3)
```

	Comp 1	Comp 2	Comp 3	Comp 4	Comp 5	Comp 6
FuelCap	-0.349	0.597	-0.127	0.681	0.203	-0.028
Weight	-0.345	0.602	-0.140	-0.698	-0.104	0.023
FrontWt	0.268	0.353	0.896	-0.014	0.033	0.001
Accel0_30	0.461	0.287	-0.219	0.182	-0.789	0.045
Accel0_60	0.486	0.193	-0.240	-0.069	0.444	0.683
QtrMile	0.487	0.196	-0.237	-0.102	0.357	-0.728

Let's check out the `plsr()` output.

```
summary(cars_pls)
Number of components considered: 6
TRAINING: % variance explained
```

	1 comps	2 comps	3 comps	4 comps	5 comps	6 comps
X	60.26	82.64	95.66	98.53	98.91	100.00
HwyMPG	53.42	66.40	68.78	71.22	71.74	71.87

```
round(cars_pls$projection, 3)
```

	Comp 1	Comp 2	Comp 3	Comp 4	Comp 5	Comp 6
FuelCap	-0.519	-0.382	0.501	0.787	0.049	-0.102
Weight	-0.600	-0.679	-0.366	-0.793	-0.031	0.032
FrontWt	0.291	0.179	0.810	-0.144	0.004	-0.028
Accel0_30	0.281	-0.400	0.006	0.182	-0.155	0.545
Accel0_60	0.319	-0.389	-0.198	-0.114	-0.619	-1.475
QtrMile	0.324	-0.373	-0.137	0.022	0.775	0.920

pls Function Output Interpretation

More Differences Between PCA and PLS

In PCA, the loading matrix comes from the eigenvectors, which are orthogonal, so if we call the loading matrix $\mathbf{P}$ (as is common in the literature), we have that $\mathbf{P}^{-1} = \mathbf{P}^T$.

We can obtain the components (also called the scores) using $\mathbf{T} = \mathbf{XP}$. That means $\mathbf{X} = \mathbf{TP}^T$. Then we regress $\mathbf{y}$ on the first k columns of $\mathbf{T}$ instead of $\mathbf{X}$. That gives us

In PLS, the loadings are not orthogonal. So, while we can still decompose $\mathbf{X}$ using $\mathbf{X} = \mathbf{TP}^T$, the components can be found using $\mathbf{T} = \mathbf{X}(\mathbf{P}^T)^{-1}$. That matrix, $(\mathbf{P}^T)^{-1}$, is sometimes called the **projection** matrix and it gives the linear combinations needed to obtain the components. For PCA, the loading and projection matrix are the same.

PLS Algorithm

Unlike PCA, the PLS projection matrix is obtained iteratively rather than using eigenvalues. Here is one way to obtain the scores:

1. Center and scale the original data matrix $\mathbf{X}$ (without an intercept column) by column. We will still call this scaled matrix $\mathbf{X}$. $\mathbf{y}$ need not be centered or scaled.
2. Obtain the first column of the project matrix, ϕ_1 , by setting the j th entry of ϕ_1 equal to the slope from the simple linear regression of $\mathbf{y}$ onto $\mathbf{X}_j$, the j th column of $\mathbf{X}$.
 - This is equivalent to taking $\phi_1 = \mathbf{X}^T \mathbf{y}$ (up to a constant).
3. Then normalize these values so the length of the vector ϕ_1 is 1.
4. Compute the first column of the scores matrix: $\mathbf{V}_1 = \sum_{j=1}^{p-1} \phi_{j1} X_j = \mathbf{X} \phi_1$.
5. Adjust each of the variables, $\mathbf{X}_1, \dots, \mathbf{X}_{p-1}$, for $\mathbf{V}_1$ by regressing each variable on $\mathbf{V}_1$ and taking the residuals.
 - These residuals can be interpreted as the remaining information that has not been explained by the first PLS direction.
6. Create a new matrix, $\mathbf{R}$ with columns equal to the residuals of each regression conducted in step 5.
7. Obtain $\tilde{\phi}_2 = \mathbf{R}^T \mathbf{y}$ and normalize it.
 - Note: this $\tilde{\phi}_2$ is not exactly equal to the second column of the projection matrix, but can be used to obtain the second column of the scores.
8. Compute the second column of the scores matrix: $\mathbf{V}_2 = \mathbf{R} \tilde{\phi}_2$.
9. Repeat steps 5-8 until all columns of the scores matrix have been obtained.

Let's implement this in **R**:

```
y <- cars$HwyMPG # y need not be scaled. It doesn't matter.
X_s <- scale(as.matrix(X)) # Scale X
R <- X_s # Initialize R matrix
# Initialize scores matrix
Scores <- matrix(rep(0, nrow(X_s)*ncol(X_s)), ncol = ncol(X_s))

for(i in 1:6) {
  phi <- t(R) %*% y # Find phi. First time through, R = X_s
  phi <- phi/sqrt(sum(phi^2)) # Normalize phi
  v <- R %*% phi # Get column of scores
  Scores[,i] <- v # Update scores matrix
  for(j in 1:ncol(X_s)) {
    R[,j] <- lm(R[,j] ~ v)$residuals # Update R matrix
  }
}
# Get projections
Proj <- solve(t(X_s) %*% X_s) %*% t(X_s) %*% Scores
# Get loadings
P <- t(solve(Proj))
```

```
round(Proj, 3)
      [,1] [,2] [,3] [,4] [,5] [,6]
FuelCap -0.519 -0.382  0.501  0.787  0.049 -0.102
Weight  -0.600 -0.679 -0.366 -0.793 -0.031  0.032
FrontWt   0.291  0.179  0.810 -0.144  0.004 -0.028
Accel0_30 0.281 -0.400  0.006  0.182 -0.155  0.545
Accel0_60 0.319 -0.389 -0.198 -0.114 -0.619 -1.475
QtrMile   0.324 -0.373 -0.137  0.022  0.775  0.920
```

Note: typically the algorithm used in software finds the loadings first and then inverts the transpose of that matrix to give the projections. More info on the algorithm can be found in the [“pls” package documentation](#).

I implemented their method in the notes in the Quarto file for these notes, but I will not show it here since the one I showed above makes more sense (to me).

PLS Surrogates

One weakness of PLS and PCA is that the components that are created are difficult to interpret in terms of the original variables. However, we can use the projection (or rotation) values to come up with combinations of our variables that are similar to the way the components are defined. These new combinations are called **surrogates** and they are often much more interpretable than the components themselves. When we define the surrogates, we ignore the variables with small weights (in magnitude). We don’t want to ignore any with a projection value around 0.3 or higher unless others are quite a bit larger. Let’s think about what they would be in this case:

Creating these surrogates in **R**:

```
X_s <- scale(as.matrix(X)) # need to scale here if we did in the plsr() function
surrogate1 <- (X_s[,1] + X_s[,2])/2 - (X_s[,3] + X_s[,4] + X_s[,5] + X_s[,6])/4
surrogate2 <- (X_s[,1] + X_s[,2] + X_s[,4] + X_s[,5] + X_s[,6])/5
surrogate3 <- (X_s[,1] + X_s[,3])/2
surrogate4 <- X_s[,1] - X_s[,2]
surrogate5 <- X_s[,6] - X_s[,5]
surrogate6 <- (X_s[,4] + X_s[,6])/2 - X_s[,5]
```

We can then use these in the model to predict values of Y , as we will soon.

Cross Validation in PLS

The `plsr()` function has an option built in for cross validation to pick the number of components. The options are “none”, “CV” for 10-fold cross validation (by default, using the `segments` option can change how many folds are used), and “LOO” for leave-one-out cross validation.

The “adjCV” is a biased-corrected CV estimate.

```
set.seed(2024)
cars_pls_cv <- plsr(HwyMPG ~ FuelCap + Weight + FrontWt +
  Accel0_30 + Accel0_60 + QtrMile,
  data = cars, scale = TRUE, validation = "CV")
summary(cars_pls_cv)
```

```
Data:   X dimension: 73 6
      Y dimension: 73 1
Fit method: kernelpls
Number of components considered: 6
```

VALIDATION: RMSEP

Cross-validated using 10 random segments.

	(Intercept)	1 comps	2 comps	3 comps	4 comps	5 comps
CV	3.085	2.180	1.895	1.841	1.799	1.848
adjCV	3.085	2.174	1.889	1.834	1.791	1.832
		6 comps				
CV		1.831				
adjCV		1.819				

TRAINING: % variance explained

	1 comps	2 comps	3 comps	4 comps	5 comps	6 comps
X	60.26	82.64	95.66	98.53	98.91	100.00
HwyMPG	53.42	66.40	68.78	71.22	71.74	71.87

Cross Validation in PLS with Caret

The `caret` package also has `pls` and `pcr` methods for partial least squares and PCA, respectively.

```
set.seed(2026)
cars_pls_train <- train(
  HwyMPG ~ FuelCap + Weight + FrontWt + Accel0_30 + Accel0_60 + QtrMile,
  data = cars, method = "pls", scale = T, tuneGrid = data.frame(ncomp = 1:6),
  trControl = trainControl(method = "cv", number = 10)
)

summary(cars_pls_train$finalModel) # It found 4 componenets was best
Data:   X dimension: 73 6
       Y dimension: 73 1
Fit method: oscorespls
Number of components considered: 4
TRAINING: % variance explained
      1 comps  2 comps  3 comps  4 comps
X      60.26   82.64   95.66   98.53
.outcome 53.42   66.40   68.78   71.22
```

Fitting Final PLS Model

```
car_surrogate_mod <- lm(
  HwyMPG ~ surrogate1 + surrogate2 + surrogate3 + surrogate4, data = cars
)
car::vif(car_surrogate_mod)
surrogate1 surrogate2 surrogate3 surrogate4
1.423053   1.673523   1.736003   1.068927
```

```
summary(car_surrogate_mod)
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  28.9726     0.1983 146.103 < 2e-16 ***
surrogate1   -1.7875     0.1528 -11.699 < 2e-16 ***
surrogate2   -2.4234     0.4767  -5.084 3.11e-06 ***
surrogate3    0.8460     0.4053   2.087 0.04060 *
surrogate4    1.1692     0.4205   2.780 0.00702 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.694 on 68 degrees of freedom
Multiple R-squared:  0.7112,    Adjusted R-squared:  0.6942
F-statistic: 41.87 on 4 and 68 DF,  p-value: < 2.2e-16
```

```
# Compare to model using actual pls components. They should be similar.
pls_df <- data.frame(comp1 = cars_pls$scores[,1], comp2 = cars_pls$scores[,2],
                     comp3 = cars_pls$scores[,3], comp4 = cars_pls$scores[,4],
                     HwyMPG = cars$HwyMPG)
car_pls_mod <- lm(HwyMPG ~ comp1 + comp2 + comp3 + comp4, data = pls_df)
car::vif(car_pls_mod)
comp1 comp2 comp3 comp4
      1      1      1      1
```

```
summary(car_pls_mod)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	28.9726	0.1980	146.352	< 2e-16 ***
comp1	1.2297	0.1095	11.234	< 2e-16 ***
comp2	0.9949	0.1797	5.538	5.34e-07 ***
comp3	0.6072	0.2558	2.374	0.0204 *
comp4	1.1586	0.4828	2.400	0.0192 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.691 on 68 degrees of freedom

Multiple R-squared: 0.7122, Adjusted R-squared: 0.6953

F-statistic: 42.07 on 4 and 68 DF, p-value: < 2.2e-16

```
summary(car_surrogate_mod) # Repeat output from surrogate model
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	28.9726	0.1983	146.103	< 2e-16 ***
surrogate1	-1.7875	0.1528	-11.699	< 2e-16 ***
surrogate2	-2.4234	0.4767	-5.084	3.11e-06 ***
surrogate3	0.8460	0.4053	2.087	0.04060 *
surrogate4	1.1692	0.4205	2.780	0.00702 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.694 on 68 degrees of freedom

Multiple R-squared: 0.7112, Adjusted R-squared: 0.6942

F-statistic: 41.87 on 4 and 68 DF, p-value: < 2.2e-16

UMAP

Another common dimension reduction technique is Uniform Manifold Approximation and Projection, more commonly known as UMAP. UMAP, unlike PCA and PLS, is a non-linear method that excels at preserving both local clusters and global relationships.

With PCA, we take a p -dimensional space and project it down to k -dimensions. It is kind of like looking at a shadow or “snapshot” of the data space from a certain perspective. This works well in some cases, but not well in others.

On the other hand, UMAP works by thinking of the data as not just being in p -dimensional space, but as sitting on a manifold: a smooth, curved surface. This is an algebraic concept. On manifolds, distances change based on where a point is. UMAP makes two mathematical assumptions:

- The manifold is locally connected.
- The distance metric is approximately locally constant.

The goal is to find a low-dimensional representation of this manifold with a similar topological structure. This is much better in cases where variables are related in a non-linear way and it’s main goal is for visualization, not modeling. How it does this is quite complicated and requires knowledge of abstract algebra and topology, so the details are omitted. But what we want to know is that there are two main parameters that affect the UMAP behavior: the number of neighbors and the minimum distance.

- The **number of neighbors** tells the UMAP how many nearby points to look at when trying to learn the structure of the data. The more neighbors, the more global the structure will be.
 - When the number of neighbors is small, UMAP focuses on very local, fine-grained patterns. You will see many small, tight clusters, but may lose the global structure of the data.
 - When the number of neighbors is large, UMAP looks at the “big picture” and tries to preserve the broad, global relationships between distant parts of the data. However, some nuances in the data can be lost.
- The **minimum distance** affects how fat apart the points are allowed to be from each other in the final plot.
 - A small minimum distance allows the points to clump together very tightly.
 - A larger minimum distance prevents points from overlapping and forces the clusters to spread out.

One more thing to note about UMAP is that the algorithm is stochastic (has random elements). That means the **UMAP is NOT deterministic**: it leads to slightly different results when running it again unless you specify a random seed or random state when running the algorithm.

Let’s check out an example of UMAP involving a woolly mammoth: [link here!](#)

UMAP in R

We can compute a UMAP in **R** using the `umap()` function in the `umap` package. Let’s consider a classic example with the built-in `iris` dataset to see if the different species separate nicely. The default values for the number of neighbors, `n_neighbors`, is 15 and the minimum distance, `min_dist`, is 0.1. These can be checked by typing `umap.defaults` once the `umap` package is loaded.

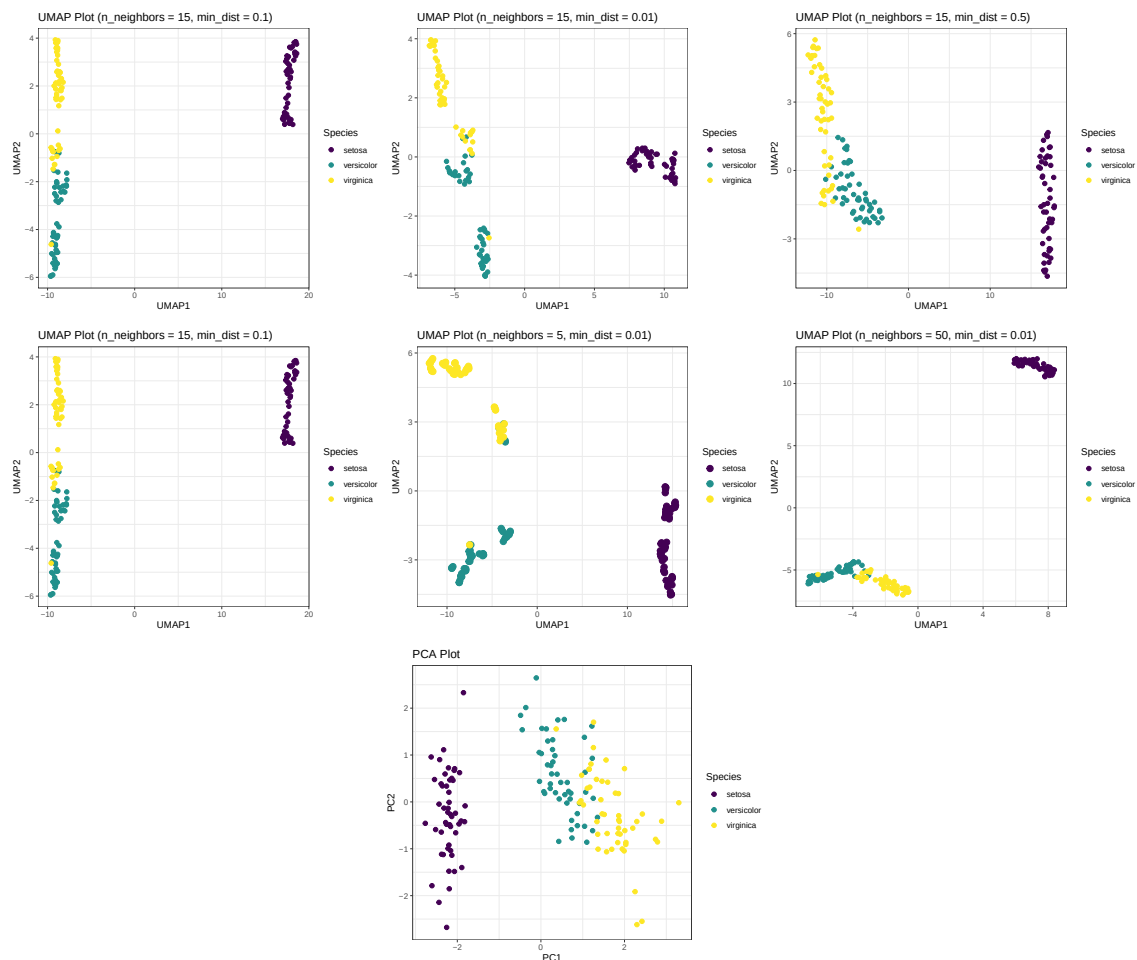
```
library(umap)
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

```
iris_umap <- umap(iris[,1:4], n_components = 2, random_state = 1,
                  n_neighbors = 15, min_dist = 0.1) # default values
iris_pca <- prcomp(iris[,1:4], scale = TRUE)$x
```

The main component of the object created by the `umap()` function is “layout”, which holds a matrix with coordinates. We can plot the 2D projection using `ggplot`. I wrote a function to do this called `umap_plot()` that is in the `math3190package`.

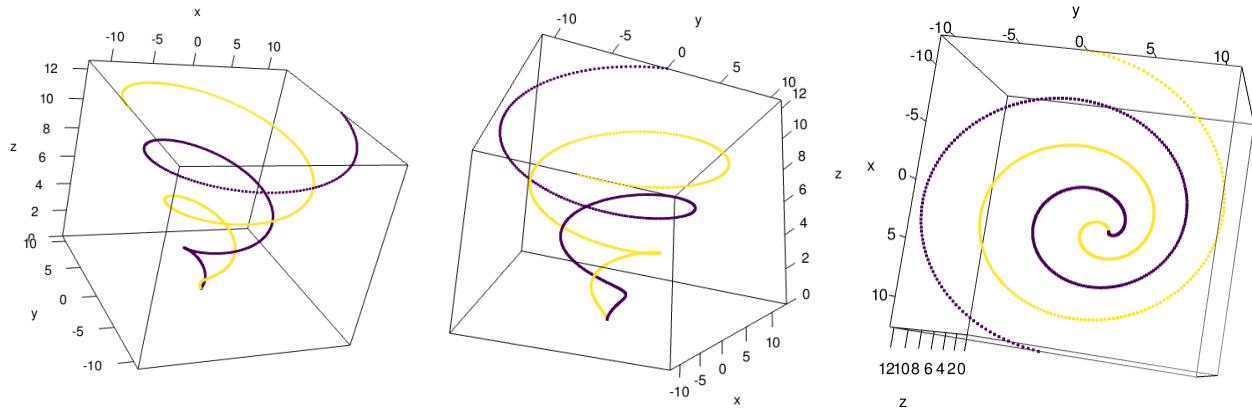
```
umap_plot(iris_umap, data = iris, color_umap = "Species", size = 2) +
  scale_colour_viridis_d() +
  labs(title = "UMAP Plot (n_neighbors = 15, min_dist = 0.1)")
```



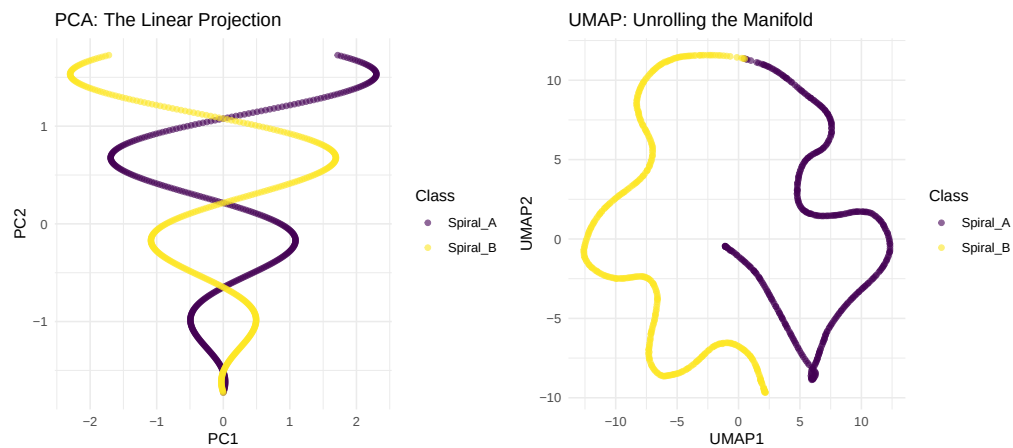
UMAP 3D Example vs PCA

Let's do one more example to highlight the strength of UMAP: non-linear associations, especially with other noise in the data.

Let's start with a nice clear 3D image of two spirals, shown from different angles below.



Let's do both a PCA and UMAP projection of this 3D image into 2D.



Now suppose instead of this nice 3D image, we had a 10-dimensional data space with 7 dimensions of noise (unrelated variables). When projecting that down to 2D, PCA can't find any signal. It just looks like a mess. However, the UMAP still finds good separation.

